C++ Contracts for C

Document #:	D4299R0 + (future WG14 N Paper)
Date:	2026-10-29
Project:	Programming Language C++
Audience:	SG22 and WG14

Reply-to: Joshua Berne <jberne4@bloomberg.net>

Abstract

C++26 has support for Contracts, introduced by [P2900R14]. Much of that facility has been designed so that C could adopt a compatible subset of that functionality to benefit the many users that use C, and more importantly the vast majority of humanity that mixes the use of C and C++ code quite liberally. This proposal describes how C support for Contracts can work, with specific names, semantics, and API choices designed for interoperability between the two languages.

Contents

1 Introduction **3**

2 Motivation **3**
- 2.1 A Primer on C++ Contract Assertions 5
- 2.2 Concerns with "C semantics for contracts" 5

3 Proposal **6**

4 Contract Assertions **6**
- 4.1 Keywords 6
- 4.2 Syntax 7
- 4.3 Postcondition Result Naming 7
- 4.4 Semantics 7

5 Headers **8**
- 5.1 `<contracts.h>` 8
- 5.2 `<stdcontracts.h>` 8

6 Violation Handling **8**
- 6.1 The `contract_violation_t` Type 8
- 6.2 The Contract-Violation Handler 9
- 6.3 Dispatch Mechanism 9

7 Accessor Functions **10**
- 7.1 Named Constants 10

8 Library API for Triggering Violations **11**
8.1 Naming Conventions 11
8.2 Convenience Macros 11
8.3 Explicit Functions 12
8.4 No Exception-Related Variants 12
8.5 Usage Example 12

9 Evaluation Semantic Configuration **13**

10 Runtime Library Placement **13**
10.1 Components 13
10.2 Implementation Strategies 14
10.2.1 Strategy A: All in the C++ standard library 14
10.2.2 Strategy B: C layers in the compiler runtime 14
10.2.3 Strategy C: Separate contracts library 14
10.3 Recommendation and Realized Approach 15

11 Implementation Experience **15**

12 Wording Changes **16**

13 Conclusion **17**

Revision History

Revision 0

- Original version of the paper

1 Introduction

NOTE: This is a very early draft of this paper, not yet ready for publication. It is available as a draft for those who want a reference for what has been implemented in the p3850 branches of GCC and clang for C support, as well as how to expect that work to behave when integrating with the C++ ABI.

The C++ Standard now includes a Contracts feature, introduced in [P2900R14] and shipped in C++26. This facility provides three forms of contract assertion — `pre`, `post`, and `contract_assert` — along with a central contract-violation handling mechanism that invokes a user-replaceable handler when a violation is detected.

Contracts in C++, notably, are almost exclusively about the use of functions and standard statements — they have almost nothing at their heart which is specific to a feature of C++ whcih is not also a feature of C. For this reason, all of the engineering decisions and principles that guided the design and evolution of Contracts in C++ are completely applicable to C, and the results of the C++ process can be readily applied to C with great benefit.

2 Motivation

Every function you, and everyone you know, has ever written has a contract. It is a plain-language contract that may exist only in the mind of the function author, but it is still there. It describes what the function's expectations are to be used, and what the function promises to do when it is used.

Functions do much more than that, because not everything a function does is promised to its clients. Functions can also do less, because thinking and intending that a function do something specific does not mean it is implemented properly. Functions that do less have what we call bugs, and finding and fixing those bugs is the purpose of *contract checking*.

A *contract checking* facility is a way for the author of a program to encode not just the algorithm that they think will implement their function but also to encode algorithms that can detect when the function's contract is not being met.

- Checks in general are assertions — they assert that something mus tbe true. Assertions don't attempt to alter that state, and should never be required to be evaluated in order for the function to satisfy its contract.
- Checks that identify misuse of a function are *precondition checks*. These indicate that someone called the function in a way or at a time that they were not supposed to call it.
- Checks that identify a when a function fails to do what it was supposed to do are *postcondition checks*.

Precondition checks are most naturally checked when a function is invoked. Sometimes it is the caller of the function that will want to check in order to be sure it is using an API corretly. Sometimes it is a function that will want to check that it is being used correctly. Sometimes it's both, and sometimes it's neither.

Postcondition checks are similar, and generally describe the state when a function returns. Again, either the caller or the callee might be interested in verifying whether the callee met its contract.

Note, however, that we haven't actually dictated that contract checks do anything — and that is becuase they are not and should not be reuqired to do anything. What a contract check does do can vary between builds of a program, and that is essential to being able to apply them usefully without harming the software they are being used to improve.

- Any check that has not been written yet does nothing, tautologically. That is the state of most programs today — they do not have checks asserting that their contracts are being met. This behavior is what we call the *ignore* evaluation semantic, and it is essential that contract checks be usable with this semantic so that they can be added to existing programs without disturbing the behavior of those programs.
- Any check can also be *enforced*. That is when we choose to run the check when it is encountered and terminate the program when the check detects a problem. This is the primary mechanism we use to benefit from our checks at runtime, quickly and loudly detecting when our programs have bugs so that we can fix them.
- A check may also be *observed*, which is equally essential to safely identify when a program that seemed to be running correctly happens to have been running with bugs. The *observe* semantic checks and then notifies the user if there was a problem, but allows the program to continue running exactly as it did when the check wasn't there. Yes, that program is running with a bug — but it ran with that bug without the check previously. The bug might even be in how the check was written, and bringing a production system down because of such bugs is rarely going to result in positive responses.
- Finally, a check may be *assumed*, where you indicate that you have so much confidence that the contract in question is not being violated that you let the compiler generate code that makes such violations *undefined behavior*. This can certainly make violations have worse consequences than *ignoring* the check, but it can lead to significant performance improvements in other cases.

Each of these semantics is an essential part of the lifecycle and use of a piece of software. Most importantly, at any given time the same piece of code will benefit from being bulit and deployed to different places — unit tests, new clients, legacy systems, performance critical hosts, etc. — with very different choices for each of these semantics.

One more piece makes a good contract checking facility highly useful, and that is giving users full control over what happens when a violation is detected. Just as program termination is often not the best or even a viable option in many domains, sending a canned compiler-generated diagnostic message to `stderr` is equally unuseful for many. Because of that, it is essential that the owner of a program get to choose what happens when a violation is detected:

- Highly secure environments might wish to terminate immediately with no leakage of information

to the surrounding environment.

- Software deployed to remote locations might need to gather and send data through logging and telemtry APIs in order for the knowledge of a failure to reach anyone that might be able to act on it.
- The common use case, similar to what happens with `assert`, of logging a diagnostic to `stderr` is sufficient in many environments — but disastrous in others.

Due to these varied needs, it is important that a contract-checking facility provides a mechanism to install a *contract-violation handler* that is invoked when contract check that is *enforced* or *observed* detects a violation.

2.1 A Primer on C++ Contract Assertions

TODO: Describe `pre post`, `contract_assert`

TODO: Describe semantics, impl-defined choice of semantics.

TODO: Describe the violation handler. how it has been designed so that a C contract-violation handler can be completely ABI-compatible, with only one (in C or C++) in a program.

TODO: Describe why not on function pointers, not on types, etc. Deployment reasons why changing types and requiring matches is awful.

2.2 Concerns with "C semantics for contracts"

Paper trail: [N3739] → [N3789] → [N3867] → [N3929].

TODO: ignore is essential

TODO: Adding contracts to existing code without breaking ABI is essential, especially for shared objects or binary-library distributions of any kind. As specified, the "aux" functions must be avialable for a program to link properly, which means any mismatches between callers and callees cannot be supported. The inlining of assumptions similarly bakes in ABI promises that result in broken programs if different library versions are linked together.

TODO: The contract-violation handler is not a security or behavioral issue, it is needed to make use of contracts at scale. Concerns about runtime-selectable callbacks are irrelevant exactly because it is not runtime-selectable.

TODO: Contracts as part of the type system are unusable

TODO: specifying the number of evaluations makes side effects dependable, which is bad

Minor issues:

- TODO: The "mapping" specified to avoid letting people specify a return value name is broken for void-returning functions
- TODO: static asserting just because an assertion is a constant expression is unmaintainable.
- TODO: "ghost state" while useful is not essential and much more involved than indicated in this proposal.

- TODO: "warranting to callres" that the postconditions hold is an ABI promise and a source of UB if contracts change.
- TODO: As specified, if any predicate *does* have side effects, or can't be proven to not have side effects, it *must* be evaluated twice. That's awful. or not. The paper is very inconsistent about whether it allows for side effects in contracts or not, and whether it makes that UB, and when it assumes they don't, allows it, or just spends time criticizing the C++ facilit for not making it UB.

Correlated concerns:

- TODO: Note that [N3930] is useless — the functions it is providing wrappers for should just be able to have _Pre and _Post put directly on them.

3 Proposal

This paper proposes that C adopt a compatible subset of that functionality, with three goals ordered by priority:

1. A C-defined contract-violation handler can handle violations originating from both C and C++ code in the same program. This is the primary goal: mixed-language programs should have a single, unified violation-handling mechanism.
2. C provides library functions equivalent to those proposed in [P3290R4] for directly triggering the contract-violation handling process. These functions allow existing C assertion facilities to integrate with the C++ contract-violation handler.
3. C provides contract-assertion keywords (`_Pre`, `_Post`, `_ContractAssert`) with semantics identical to their C++ counterparts, enabling C code to express preconditions, postconditions, and assertions using the same evaluation semantics and configuration mechanisms as C++.

The remainder of this paper describes each component of the proposal, the specific names and signatures chosen, and the rationale behind each design decision.

4 Contract Assertions

4.1 Keywords

C introduces three new keywords for contract assertions:

- `_Pre(`*conditional-expression*`)` — precondition
- `_Post(`*identifier*`:` *conditional-expression*`)` — postcondition with named result
- `_Post(`*conditional-expression*`)` — postcondition without result access
- `_ContractAssert(`*conditional-expression*`)` — assertion statement

The predicate is a *conditional-expression*, as in C++: a comma expression or an assignment must be parenthesized.

These names follow the C convention of introducing new keywords with a leading underscore and uppercase letter, as was done for `_Bool`, `_Static_assert`, `_Alignof`, `_Noreturn`, and `_Thread_local` in C11.

4.2 Syntax

Contract specifiers appear after the closing parenthesis of the parameter list, before the function body or semicolon. This position mirrors where C++ places `pre` and `post` and aligns with where GCC attributes are placed after a parameter list.

```
// Declaration with precondition and postcondition
int foo(int x) _Pre(x > 0) _Post(r: r > 0);

// Definition with precondition
void bar(int* p) _Pre(p != NULL) {
  _ContractAssert(*p > 0);
  /*...*/
}

// Postcondition without result access
void reset(int* p) _Pre(p != NULL) _Post(*p == 0) {
  *p = 0;
}
```

4.3 Postcondition Result Naming

In C++, a postcondition names its result value using the syntax `post(r: r > 0)`. C uses an identical syntax: `_Post(r: r > 0)`. The *identifier*`:` prefix introduces a name for the function's return value, scoped to the postcondition predicate expression.

This syntax requires a new grammar production inside `_Post(...)` but is no more invasive than the keyword itself. The parser distinguishes *identifier*`:` from the ternary operator `?:` by checking for an identifier followed by `:` at the start of the parenthesized expression.

4.4 Semantics

The semantics of C contract assertions are identical to C++26 as specified in [P2900R14]. Contract predicates are evaluated per the configured evaluation semantic:

- *enforce* — evaluate the predicate; on violation, invoke the contract-violation handler and then terminate the program
- *observe* — evaluate the predicate; on violation, invoke the contract-violation handler and then continue execution
- *quick-enforce* — evaluate the predicate; on violation, terminate the program immediately, without invoking the handler. The means of termination (for example an illegal-instruction trap or a call to `abort`) is implementation-defined.
- *ignore* — do not evaluate the predicate at all

5 Headers

Two headers are provided:

5.1 `<contracts.h>`

This header provides the violation-handling API and is safe to include without introducing name conflicts. It declares:

- The opaque type `contract_violation_t`
- Accessor functions for inspecting a violation (`stdc_contract_violation_*`)
- Named constants for assertion kind, evaluation semantic, and detection mode
- The library API for triggering violations (`stdc_handle_*_contract_violation`)
- The handler declaration pattern
- The macro `__STDC_VERSION_CONTRACTS_H__`, following the C23 convention that each standard header defines a `__STDC_VERSION_`*name*`_H__` macro giving its revision

When included in C++, `<contracts.h>` provides the C API functions and types without interfering with the C++ `<contracts>` header.

5.2 `<stdcontracts.h>`

This header provides convenience macros that map lowercase names to the underscore-prefixed keywords:

```
#include <contracts.h>
#ifndef __cplusplus
#define pre            _Pre
#define post           _Post
#define contract_assert _ContractAssert
#endif
```

When included in C++, this header does nothing, since C++ already has `pre`, `post`, and `contract_assert` as keywords.

This header exists separately from `<contracts.h>` because `pre` and `post` are extremely common identifiers in existing C code. Only code that explicitly includes `<stdcontracts.h>` gets these macros, following the precedent set by `<stdbool.h>` providing the `bool` macro for `_Bool` in C11.

6 Violation Handling

6.1 The `contract_violation_t` Type

The header `<contracts.h>` declares `contract_violation_t` as a completely opaque type:

```
typedef struct contract_violation_t contract_violation_t;
```

The struct is never defined in any C-visible header. C clients cannot determine its size, access its members, or do anything other than pass pointers to it to the accessor functions described in §7.

Internally, a `const contract_violation_t*` points to the very object a C++ handler receives as `std::contracts::contract_violation` — a thin handle whose only state is the head of the `__cxa_contract_data_bloc` chain that describes the violation. The C accessor functions of §7 read that chain directly; C code need not be aware of any of this.

6.2 The Contract-Violation Handler

A C translation unit can define a contract-violation handler:

```
#include <contracts.h>
#include <stdio.h>

void handle_contract_violation(
    const contract_violation_t* cv)
{
  fprintf(stderr, "violation: %s at %s:%u\n",
          stdc_contract_violation_comment(cv),
          stdc_contract_violation_file(cv),
          stdc_contract_violation_line(cv));
}
```

A program may contain at most one definition of `handle_contract_violation`, whether it is defined in C or in C++. When a C compiler compiles a definition of this function, it emits an alias named `__handle_contract_violation`, the same alias that a C++ compiler emits for the C++ handler `::handle_contract_violation(const std::contracts::contract_violation\&)`.

The implementation enforces that a C definition of `handle_contract_violation` has the required signature: return type `void` and a single parameter of type `const contract_violation_t*`. A definition with any other signature is diagnosed as an error. This corresponds to the C++ requirement that the replacement function satisfy the signature requirements of the contract-violation handler.

6.3 Dispatch Mechanism

The runtime dispatch path is identical regardless of whether the handler is written in C or C++:

1. A contract violation is detected (in either C or C++ code).
2. The `__cxa_contract_violation_*` ABI entry point assembles the `__cxa_contract_data_block` chain describing the violation and forms the handle (*ptr*) to it. No C++ object is created on this path.
3. The entry point calls `__handle_contract_violation(`*ptr*`)`.
4. The handler receives the pointer — as `const contract_violation_t*` in C, or as a reference to `contract_violation` in C++.

At the ABI level, a C++ reference parameter and a C pointer parameter are both passed as a pointer, so the `__handle_contract_violation` symbol works identically for handlers defined in either language. The runtime has no knowledge of which language the handler is defined in.

This design also composes with C++ local violation handlers. When a C++ contract assertion has a local handler (e.g., via a P3400 label facet), the local handler is invoked first with the `contract_violation` object. If the local handler returns `NOT_HANDLED`, the *same* object is then passed to the global handler — whether that global handler is defined in C or C++.

7 Accessor Functions

The following functions are declared in `<contracts.h>` for inspecting a `contract_violation_t`. All use the `stdc_` prefix, following the naming convention introduced by C23's `<stdbit.h>` header (e.g., `stdc_bit_ceil`, `stdc_count_ones`).

```
const char* stdc_contract_violation_comment(
    const contract_violation_t* cv);
const char* stdc_contract_violation_file(
    const contract_violation_t* cv);
const char* stdc_contract_violation_function(
    const contract_violation_t* cv);
unsigned stdc_contract_violation_line(
    const contract_violation_t* cv);
unsigned stdc_contract_violation_column(
    const contract_violation_t* cv);
int stdc_contract_violation_kind(
    const contract_violation_t* cv);
int stdc_contract_violation_semantic(
    const contract_violation_t* cv);
int stdc_contract_violation_detection_mode(
    const contract_violation_t* cv);
int stdc_contract_violation_is_terminating(
    const contract_violation_t* cv);
```

These functions correspond directly to the member functions of `std::contracts::contract_violation` in C++.

7.1 Named Constants

The `kind`, `semantic`, and `detection_mode` accessors return integer values. `<contracts.h>` provides named constants for these values:

```
// Assertion kind
#define STDC_CONTRACT_PRE            0x01
#define STDC_CONTRACT_POST           0x02
#define STDC_CONTRACT_ASSERT         0x03
#define STDC_CONTRACT_MANUAL         0x04
#define STDC_CONTRACT_CASSERT        0x05
#define STDC_CONTRACT_POST_CAPTURE   0x06
#define STDC_CONTRACT_IMPLICIT       0x07

// Evaluation semantic
#define STDC_CONTRACT_IGNORE         0x01
#define STDC_CONTRACT_OBSERVE        0x02
```

```
#define STDC_CONTRACT_ENFORCE        0x03
#define STDC_CONTRACT_QUICK_ENFORCE  0x04

// Detection mode
#define STDC_CONTRACT_UNSPECIFIED          0x00
#define STDC_CONTRACT_PREDICATE_FALSE      0x01
#define STDC_CONTRACT_EVALUATION_EXCEPTION 0x02
```

The `MANUAL` kind is used for violations triggered via the library API described in §8. The `CASSERT` kind is used for violations originating from the `assert` macro when it is configured to use Contracts. The `POST_CAPTURE` and `IMPLICIT` kinds do not arise from C contract assertions — C supports neither postcondition captures nor implicit (language-level) contract assertions — but a shared handler in a mixed C and C++ program can receive a violation of either kind from the C++ side, so the constants are provided for a C handler to recognize them. The `EVALUATION_EXCEPTION` detection mode is included for completeness but will not arise from C code, as C does not have exceptions.

C++ proposals add further evaluation semantics: `assume` (P3100) and the nonthrowing `noexcept_observe` and `noexcept_enforce` (D4298). None of them is part of this proposal — the nonthrowing semantics exist to stop an exception escaping a violation handler, and C has no exceptions — but an implementation may support them for C contract assertions as an extension, and a handler in a mixed C and C++ program can receive them from the C++ side. An implementation that provides them should use these names and values, which match `std::contracts::evaluation_semantic`:

```
#define STDC_CONTRACT_ASSUME           0x05
#define STDC_CONTRACT_NOEXCEPT_OBSERVE 0x06
#define STDC_CONTRACT_NOEXCEPT_ENFORCE 0x07
```

These names are not required by this proposal.

8 Library API for Triggering Violations

C++ provides library functions for directly triggering the contract-violation handling process, as proposed in [P3290R4]. This section describes the C equivalents.

8.1 Naming Conventions

Since C does not support function overloading, each variant requires a distinct name. Two naming conventions from existing C standards inform the design:

- The `stdc_` prefix, introduced in C23's `<stdbit.h>` header, is used for all function names. This prefix marks functions as part of the C standard library and avoids collisions with user code.
- The `_explicit` suffix, used by C11's `<stdatomic.h>` (e.g., `atomic_store` vs. `atomic_store_explicit`), distinguishes convenience macros that automatically capture context from functions that accept explicit parameters.

8.2 Convenience Macros

The following macros automatically capture the caller's source location using `__FILE__`, `__func__`, and `__LINE__`:

```
stdc_handle_enforced_contract_violation(comment)
stdc_handle_observed_contract_violation(comment)
stdc_handle_quick_enforced_contract_violation(comment)
```

The *enforced* and *quick-enforced* variants do not return. The *observed* variant returns normally after the handler completes.

8.3 Explicit Functions

Each macro expands to a call to the corresponding `_explicit` function:

```
_Noreturn void
stdc_handle_enforced_contract_violation_explicit(
    const char* comment, const char* file,
    const char* func, unsigned line);

void
stdc_handle_observed_contract_violation_explicit(
    const char* comment, const char* file,
    const char* func, unsigned line);

_Noreturn void
stdc_handle_quick_enforced_contract_violation_explicit(
    const char* comment, const char* file,
    const char* func, unsigned line);
```

Each `_explicit` function constructs the violation data with `kind` set to `STDC_CONTRACT_MANUAL`, the appropriate evaluation semantic, and `detection_mode` set to `STDC_CONTRACT_UNSPECIFIED`, then triggers the contract-violation handling process. The *quick-enforced* variant terminates the program immediately without invoking the handler, matching the behavior of the *quick-enforce* evaluation semantic.

8.4 No Exception-Related Variants

The C++ API in [P3290R4] provides `noexcept` overloads (accepting a `std::nothrow_t` first parameter) that invoke `std::terminate` if the handler throws an exception. Because C does not have exceptions, these variants are not needed and are not provided.

8.5 Usage Example

```
#include <contracts.h>

void legacy_precondition_check(int x, const char* msg) {
  if (x <= 0) {
    stdc_handle_enforced_contract_violation(msg);
    // Does not return
  }
}

void explicit_location_example(const char* file,
```

```
                                const char* func,
                                unsigned line) {
  stdc_handle_observed_contract_violation_explicit(
      "manual check failed", file, func, line);
  // Returns after handler completes
}
```

9 Evaluation Semantic Configuration

C contract assertions use the same evaluation-semantic configuration mechanisms as C++. An implementation provides a means to select the evaluation semantic (e.g., via a command-line flag), and the same selection applies to both C and C++ contract assertions within a translation unit.

When fine-grained configuration is available (as in the configuration-file mechanism described in the C++ Contracts extensions), the following properties are applicable to C contract assertions:

- *kind* — `pre`, `post`, or `assert`
- *location* — source file and line of the assertion

C contract assertions are always evaluated on the callee side, in the function body; C provides no caller-side checking of preconditions.

The following properties from C++ are not applicable to C and are not used when resolving the evaluation semantic for a C contract assertion:

- *namespace* — C does not have namespaces
- *constexpr* — not applicable to C contract evaluation
- *caller vs. callee side* — C contract checks are always evaluated on the callee side

10 Runtime Library Placement

A C program that uses contract assertions requires a runtime library to provide the dispatch and handler mechanisms. The placement of these functions in the implementation's library hierarchy determines what link flags, if any, users must supply when compiling C programs with contracts.

10.1 Components

The contracts runtime divides into three layers, distinguished by whether they inherently depend on C++:

1. *Dispatch and entry points* — where compiler-generated calls land. For C these are `__c_contract_check_enforce`, `__c_contract_check_observe`, and so on; shared with C++ are the `__cxa_contract_violation_*` entry points and the chain-walking primitive `__cxa_find_field`. They assemble the `__cxa_contract_data_block` chain that describes a violation and invoke `__handle_contract_violation`.
2. *Accessor and trigger API* — the `stdc_contract_violation_*` accessors a handler calls to inspect a violation, and the `stdc_handle_*` trigger functions of §8. These read the data-block chain directly.

3. *C++-specific services* — the `contract_violation` member functions, the default *diagnostic* handler, the `noexcept` entry-point wrappers that turn an escaping exception into `std::terminate`, and the `assert`-integration entry point of [P3290R4]. These genuinely require the C++ runtime.

The crucial observation is that layers 1 and 2 have *no* dependency on C++: they operate entirely on the opaque `__cxa_contract_data_block` chain — a plain C struct — and never on a C++ class object. Detecting, dispatching, and inspecting a violation from C thus needs nothing from the C++ runtime; layer 3 only adds the default diagnostic text, exception safety at `noexcept` boundaries, and the C++-side accessors.

10.2 Implementation Strategies

10.2.1 Strategy A: All in the C++ standard library

All three layers live in `libstdc++` (GCC) or `libc++` (Clang). C programs must link `-lstdc++` or `-lc++`.

This is the simplest implementation because the accessor functions can directly cast the opaque pointer to the C++ `contract_violation` class and delegate to its member functions.

The downside is that C users must link a C++ library — there is no precedent for this in either GCC or Clang for any existing C standard library feature.

10.2.2 Strategy B: C layers in the compiler runtime

Layers 1 and 2 are implemented in the compiler's automatically-linked runtime library (e.g., `libgcc` for GCC). Layer 3 remains in the C++ standard library.

The dispatch functions construct the data block directly (the `__cxa_contract_data_block` layout is a fixed C struct) and call `__handle_contract_violation` without creating any C++ objects. The accessor functions read fields from the data block pointer.

This eliminates the need for `-lstdc++` in C programs. The C++ `__cxa_contract_violation_*` entry points continue to live in `libstdc++`/`libc++` and are used only when a C++ translation unit triggers a violation.

The tradeoff is a slightly larger implementation surface: the data block layout becomes a cross-library ABI contract between `libgcc` and `libstdc++`, rather than an internal detail of `libstdc++`.

10.2.3 Strategy C: Separate contracts library

A dedicated `libcontracts` contains all three layers. Both C and C++ programs link it explicitly (`-lcontracts`).

This isolates the contracts runtime from both the C and C++ standard libraries, simplifying ABI stability guarantees. However, it introduces a new library that all programs using contracts must remember to link.

10.3 Recommendation and Realized Approach

The prototype implements a refinement of Strategy C. A dedicated, pure-C `libcontracts` holds layers 1 and 2 in full — the dispatch and `__c_contract_check_*` helpers, the shared `__cxa_contract_violation_*` entry points, the chain-walking `__cxa_find_field`, the `stdc_` accessors, and the `stdc_handle_*` trigger API — while the C++-specific layer 3 services stay in the C++ runtime (`libstdc++exp` for GCC, `libc++` for Clang). Because layers 1 and 2 touch only the C data block, `libcontracts` is libc-only and is built from a single set of C sources shared by both compilers.

Two properties make this the best of the three strategies rather than merely one of them:

- *Implicit linkage.* Enabling C contracts (`-fcontracts-p4299`) makes the driver link `libcontracts` automatically, exactly as the compiler runtime is linked for any other compiler-emitted call. This removes Strategy C's only drawback — that users must remember to link `-lcontracts` — while keeping Strategy B's "no user action" property without hiding the C code inside a compiler-private library.
- *No C++ runtime for C.* Because the C-relevant layers have no C++ dependency, a program that uses contracts only from C links `libcontracts` alone (which depends only on libc). This avoids Strategy A's requirement that C users link a C++ library, for which there is no precedent. A minimal libc-only default handler in `libcontracts` prints a diagnostic when neither a user handler nor the C++ runtime is present, so a pure-C program is fully functional on its own.

When C and C++ are mixed, the C++ runtime is linked anyway for the rest of C++, and its richer default handler and `contract_violation` members take over; `libcontracts` and the C++ runtime share the one data-block ABI, so a violation raised in either language is dispatched and inspected identically. §11 reports on this in practice.

11 Implementation Experience

C contract support has been implemented in the P3850 branches of both GCC and Clang and is available on Compiler Explorer: https://godbolt.org/z/hfreecaqz. Both implementations cover the full proposal — the `_Pre`, `_Post`, and `_ContractAssert` keywords (including the `_Post(`r`:` *predicate*`)` result-naming form), the `stdc_contract_violation_*` accessors, the `stdc_handle_*` trigger API, and a C-defined `handle_contract_violation` — and, most importantly, deliver the cross-language interoperability that is this paper's primary goal.

The compiler side was deliberately thin. C contract assertions reuse the same front-end machinery as their C++ counterparts: the keywords are late-parsed after the parameter list, the predicate is validated through the same contextual-conversion-to-`bool` path, and the resolved evaluation semantic flows through the same configuration mechanism (§9). The only genuinely new front-end work was registering the three underscore-prefixed keywords behind `-fcontracts-p4299` and, when a C translation unit defines `handle_contract_violation`, validating its signature (return `void`, a single `const contract_violation_t*` parameter) and emitting the shared `__handle_contract_violation` alias — the same alias a C++ handler produces. The two compilers differ only in incidental detail: GCC gates the keywords with a `D_C_CONTRACTS` disable mask and emits the alias from `c-decl.cc`, while Clang shares a single `KEYCONTRACTS` flag with its C++ contract keywords and emits the alias from `CodeGenModule`.

The runtime side is where the interoperability is realized, and it follows §10: the C-relevant layers live in a dedicated, pure-C `libcontracts`. The same C sources — the dispatch core and `__cxa_contract_violation_*` entry points, the chain-walking `__cxa_find_field`, the `stdc_` accessors and trigger API, and the `__c_contract_check_*` helpers — are compiled into `libcontracts` by both GCC and Clang, which export an identical `LIBCONTRACTS_1.0` symbol set. A violation raised by C code compiled with one compiler is dispatched and inspected correctly by a handler — C or C++ — built with the other: a GCC-compiled C contract program links and runs against Clang's `libcontracts`, and vice versa. This is verified with a cross-product test that pairs C, C++, and mixed contract translation units with C and C++ violation handlers across both compilers; every combination reports its violations through the one shared handler.

Linkage requires no user action. Compiling with `-fcontracts-p4299` makes the driver add `libcontracts` to the link automatically, just as it links the compiler runtime for any other emitted call. A program that uses contracts only from C therefore links nothing but `libcontracts` (which depends only on libc); the minimal libc-only default handler built into `libcontracts` prints a diagnostic when no user handler and no C++ runtime are present, so such a program is complete on its own. When C++ is also in the link, its default handler and `contract_violation` members are used instead, over the same data block. Notably, `-fcontracts-p4299` — unlike the C++ per-paper contract flags — does not enable the C++ `-fcontracts` feature and does not pull in the C++ experimental runtime; it links `libcontracts` and nothing more.

The initial level of effort was low in both compilers, precisely because C contracts are mostly a reuse of the existing C++ contract front end plus a thin, shared C runtime. One limitation remains, on the Clang side only: a `_Post` on a `void`-returning function with no explicit `return` statement does not yet have its check injected before the implicit return, where GCC handles this through a `finish_function` hook. `_Post` on a non-`void` function, and on a `void` function with an explicit `return;`, is unaffected.

12 Wording Changes

The wording changes for the proposal are, of course, for the C Standard and not the C++ Standard. They will be coming later.

These wording changes are relative to the ISO/IEC 9899 draft sources with git hash 19cd0ef2, last modified on Fri, 26 Jun 2026 18:52:58 +0200.

Modified Section Contents

6 Language **[lang-language] 16**
6.4 Lexical elements [lang-lex-lexical-elements] 17
6.4.2 Keywords [lang-lex-keywords] 17

Modifications

6 **Language** **[lang-language]**

6.4 **Lexical elements** **[lang-lex-lexical-elements]**

6.4.2 **Keywords** **[lang-lex-keywords]**

Modify section 6.4.2[lang-lex-keywords], paragraph 1:

Syntax

1 *keyword*: one of

alignas	double	nullptr	true	_ContractAssert
alignof	else	register	typedef	_Countof
auto	enum	restrict	typeof	_Decimal128
bool	extern	return	typeof_unqual	_Decimal32
break	false	short	union	_Decimal64
case	float	signed	unsigned	_Generic
char	for	sizeof	void	_Noreturn
const	goto	static	volatile	_Post
constexpr	if	static_assert	while	_Pre
continue	inline	struct	_Atomic	
default	int	switch	_BitInt	
do	long	thread_local	_Complex	

13 Conclusion

This paper describes how C can adopt a subset of the C++26 Contracts facility that provides full interoperability between the two languages. The design prioritizes a single, shared contract-violation handler that works identically for violations originating in either language. Library API functions for triggering violations manually follow established C naming conventions (`stdc_` prefix, `_explicit` suffix pattern), and the contract-assertion keywords follow the C convention of underscore-prefixed reserved identifiers.

Acknowledgments

Bibliography

[N3739] Daniel Krügler, “Improving pair and tuple, revision 1”, 2013
http://wg21.link/N3739

[N3789] Marshall Clow, “Constexpr Library Additions: functional”, 2013
http://wg21.link/N3789

[N3867] Mike Spertus, “Specializations and namespaces (Rev. 2)”, 2014
http://wg21.link/N3867

[N3929] Gustedt, “C semantics for contracts v. 4”, 2026

[N3930] Gustedt, “Interfaces with contracts alongside Annex K”, 2026

[P2900R14] Joshua Berne, Timur Doumler, and Andrzej Krzemieński, “Contracts for C++”, 2025
http://wg21.link/P2900R14

[P3290R4] Joshua Berne, Timur Doumler, and John Lakos, “Integrating Existing Assertions With Contracts”, 2026
http://wg21.link/P3290R4